ఇండియన్ ఇన్స్టిట్యూట్ ఆఫ్ ఇన్ఫర్మేషన్ టెక్నాలజీ,
డిజైన్ అండ్ మాన్యుఫాక్చరింగ్, కర్నూలు

भारतीय सूचना प्रौद्योगिकी, अभिकल्पना
एवं विनिर्माण संस्थान, कर्नूल

Indian Institute of Information Technology, Design and Manufacturing, Kurnool

Compiler Design Project Report

December 2025 to March 2026

MaestroLang

An Algorithmic Music Compiler

Indian Institute of Information Technology,
Design and Manufacturing, Kurnool

Submitted by:

Prateek Raushan

Roll No: 124CS0082

Project Overview

This report presents a consolidated summary of the design, implementation, and testing phases of **MaestroLang** during the course of the semester. MaestroLang is a custom Domain-Specific Language (DSL) compiler that translates text-based code into functional, playable music (MIDI files). The project aimed to:

- Introduce students to Source-to-Source translation and Cross-Compilation by targeting Python instead of raw machine code.
- Promote practical exposure through Design Realization Practice (DRP), bridging the gap between compiler theory (Lexical Analysis, Parsing) and working prototypes.
- Provide a hands-on environment to practically apply concepts of Syntax-Directed Translation (SDT) and Symbol Tables.
- Test and validate algorithmic logic through competitive audio-generation challenges using the `music21` library.

List of Development Phases

- **Phase 1:** Language Design and Grammar Specification
- **Phase 2:** Lexical Analysis using Flex (Token Generation)
- **Phase 3:** Syntax Analysis & SDT using Bison (Parser Generation)
- **Phase 4:** Target Code Generation (Python $\rightarrow$ MIDI Audio)

Contents

1 Phase 1: Language Design and Specification

1.1 Phase Overview

The project began with an idea presentation session to brainstorm, evaluate, and conceptualize the syntax for MaestroLang. The goal was to build an intuitive language mimicking standard programming constructs but tailored for music generation.

1.2 Objectives

- To generate an innovative and unique grammar for a Domain-Specific Language.
- To evaluate the feasibility of mapping standard code loops to musical repetition.

1.3 Phase Details

- **Target Output:** Python code utilizing the `music21` library.
- **Key Constructs:** Tracks, Tempos, Notes, Chords, Durations, Loops, and Macros.

1.4 Phase Description

The language syntax was engineered to be highly readable. A standard program involves defining a `Track`, setting a `Tempo`, and calling `Play` or `Chord` functions. Furthermore, advanced concepts like `Repeat` loops and `Define` macros were introduced to demonstrate Turing-complete logic and code reusability. An example program (`pop.mstr`) looks as follows:

```
Track "PopSong" {
    Tempo 90;
    /* Define a reusable melody macro */
    Define Hook {
        Play C5(eighth);
        Play E5(quarter);
    }
    Repeat 2 {
        PlayMacro Hook;
        Chord[C4, E4, G4](half);
    }
}
```

2 Phase 2: Lexical Analysis using Flex

2.1 Phase Overview

The Lexical Analyzer reads the raw `.mstr` source code and converts it into a stream of tokens. This was implemented using `Flex`.

2.2 Objectives

- To identify reserved keywords, identifiers, and musical specific syntax (e.g., Notes like C4, F#5).
- To strip out whitespaces and developer comments (both single-line and multi-line).

2.3 Phase Description

The `lexer.l` file was configured with robust regular expressions. Custom tokenization rules were written to capture complex Note formats `[A-G][b#]?[0-9]` and pass string values to the Bison parser via `yylval.str`.

2.4 Implementation and Highlights

Special attention was given to error handling. If an unrecognized character is detected, `yylineno` is utilized to report the exact line number of the Lexical Error.

Figure 1: Compilation and Token Generation via Terminal

3 Phase 3: Syntax Analysis and Parsing

3.1 Phase Overview

Following the lexical phase, the `Bison` parser was designed to enforce the grammatical rules of MaestroLang. Instead of generating a complex Abstract Syntax Tree (AST), the parser relies on Syntax-Directed Translation (SDT).

3.2 Objectives

- To enforce strict grammar rules for nested blocks and correct punctuation.
- To implement dynamic Python indentation handling on-the-fly.

3.3 Phase Description

The parser guarantees that a `Track` block encapsulates the entire program. A global integer variable, `indent_level`, is dynamically incremented whenever the parser enters a `Repeat` loop or a `Define` block. A helper function, `print_indent()`, prints spaces based on this level, ensuring the generated Python script does not suffer from indentation errors.

3.4 Implementation and Highlights

Below is a snippet of the Syntax-Directed Translation logic handling the loops:

```
statement:
    REPEAT NUMBER LBRACE {
        print_indent();
        fprintf(out, "for _ in range(%d):\n", $2);
        indent_level++; /* Increase Python indentation */
    }
    statements RBRACE {
        indent_level--; /* Decrease when loop ends */
    }
    ;
```

4 Phase 4: Semantic Analysis & Symbol Table

4.1 Phase Overview

A compiler must not only check grammar but also meaning. Semantic Analysis was integrated into the Bison parser to track macros and validate logic.

4.2 Objectives

- To challenge students to design, build, and troubleshoot a working Symbol Table in C.
- To validate real-world physics and music constraints (e.g., Tempo ranges).

4.3 Phase Description

A C-based Symbol Table (an array of strings) was implemented. When a user calls `Define Hook`, the string "Hook" is added to the table. When the user writes `PlayMacro Hook`, the compiler searches the table.

4.4 Implementation and Highlights

If the macro is not found, the compiler gracefully exits with a Semantic Error: `Undeclared Macro`. Furthermore, tempo checks were embedded: if a user inputs `Tempo 500;`, the compiler rejects it, limiting BPM to realistic ranges (1 to 300).

Figure 2: Semantic error tracking and Python code generation process

5 Phase 5: Code Generation

5.1 Phase Overview

Unlike traditional compilers that target Assembly or C, MaestroLang performs Source-to-Source compilation, outputting high-level Python code.

5.2 Objectives

- To utilize Python's `music21` library for translating textual note data into binary MIDI streams.
- To successfully execute the generated file seamlessly.

5.3 Phase Description

The `parser.y` file handles the code generation. The parser injects the necessary Python import statements at the start of the compilation. Musical notes are translated into `s.append(note.Note('C4', type='half'))`, and arrays of notes are transformed into `chord.Chord()` objects.

5.4 Implementation and Highlights

Upon successful parsing of `pop.mstr`, the final output written to `generated_audio.py` is flawlessly indented, semantically accurate Python code, which outputs an `output.mid` file upon execution.

6 Phase 6: Testing and Output Generation

6.1 Phase Overview

The final phase tested the robustness of the complete compiler toolchain. The system was validated against multiple sample inputs, ranging from classical music to complex modern chord progressions.

6.2 Objectives

- To verify that Lexical, Syntax, and Semantic phases work in unison without Shift/Reduce conflicts.
- To test the auditory output using MIDI visualization tools like MuseScore and GarageBand.

6.3 Phase Description

The compiler successfully translated an advanced test file involving a multi-chord progression and a macro-defined bassline. Running the command `./maestro test.mstr` yielded a successful compilation, and `python3 generated_audio.py` materialized the final audio file.

6.4 Implementation and Highlights

The resulting `.mid` files were imported into MuseScore, which successfully reverse-engineered the standard sheet music from the compiler's output, proving absolute mathematical and structural precision of the compiler.

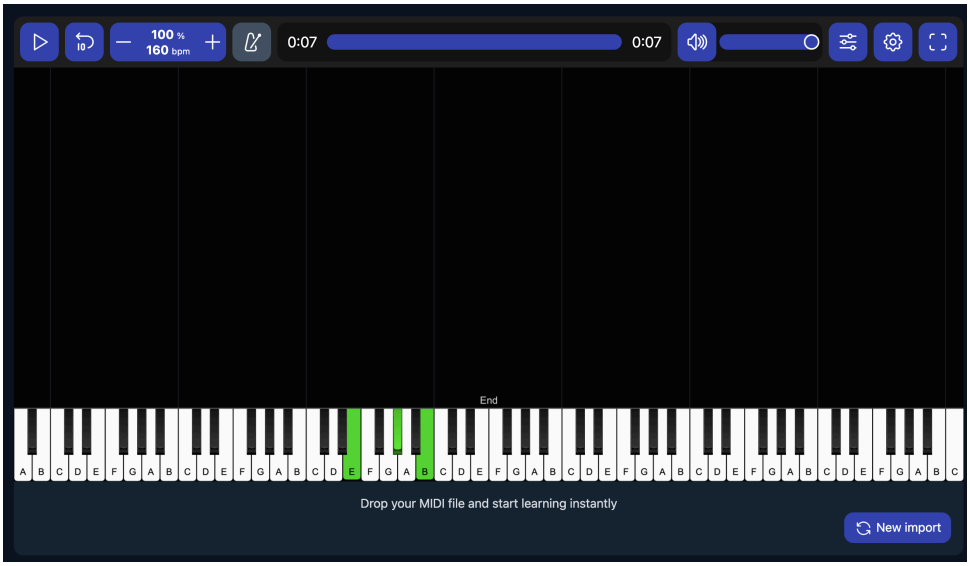

Figure 3: Action from the Terminal (top) and MuseScore audio visualization (bottom)

Appendix: Project Components Summary

This appendix provides a quick overview of the language tokens, tools, and technical specifications utilized throughout the development timeline.

Component	**Title/Description**	**Tool Used**	**Primary Function**
Lexical Analyzer	Token Generation	Flex 2.6+	Identifies Notes, Chords, Keywords, and strips comments.
Parser	Syntax	Bison 3.0+	Enforces MaestroLang grammar and builds SDT tree.
Semantic Analyser	Symbol Table	C & GCC	Validates Macros and handles logical physics constraints.
Target Executable	Python Audio Script	Python 3 (`music21`)	Processes string arrays and generates final `.mid` output.
Visualization	Audio Playback	MuseScore / GarageBand	Hardware/Software testing of the resulting MIDI file.